Engineer to Excel

SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Accredited by NAAC A Grade

SIMATS ENGINEERING

Descriptive Written Exam

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 50%

Sl. No	Questions	Bloom's Level
1	A banking application team is facing frequent failures due to unstructured testing practices. As a test engineer, apply the concept of testing as an engineering activity to redesign the testing approach. Explain how systematic test design, tools, and standards can be introduced. Illustrate how this approach improves defect detection and quality. Suggest suitable techniques to replace ad-hoc testing. Justify your solution with a practical example.	3
2	An ATM system is producing inconsistent results due to lack of a defined testing process. As a tester, demonstrate how you would implement a structured testing process across SDLC phases. Explain each stage and its relevance to quality improvement. Apply your approach to ATM withdrawal functionality. Discuss how process consistency reduces defects. Conclude with expected outcomes.	3
3	A web application shows mismatch between expected and actual outputs during login and transaction operations. Apply basic testing definitions to identify errors, defects, and failures in this scenario. Illustrate their relationships with suitable examples. Explain how improper understanding of these terms affects testing. Suggest how to improve defect reporting clarity. Justify your answer with examples.	3
4	A software project team observes that most defects are concentrated in a few modules, leading to delays. Apply software testing principles to address this issue. Explain how defect clustering and other principles guide test planning. Illustrate how test cases can be prioritized effectively. Discuss how applying these principles improves testing efficiency. Provide a real-time example.	3

5	A company maintains a defect repository but fails to utilize it effectively, resulting in repeated defects across releases. As a tester, apply defect classification and repository concepts to improve the situation. Explain how defects originate and how they can be categorized. Illustrate how repository data can support test design and prevention. Suggest improvements to avoid recurring defects. Conclude with expected quality improvements.	3
6	A mobile banking application frequently crashes during fund transfer operations. As a tester, apply software testing principles to identify the root cause of failures. Explain how systematic testing can reduce such issues. Illustrate your approach with suitable examples. Suggest improvements in testing practices to avoid future failures.	3
7	A development team confuses debugging with testing, leading to ineffective defect resolution. Apply the concepts of testing and debugging to differentiate both processes. Explain their roles in software quality. Illustrate how improper usage affects defect detection. Suggest a structured approach to integrate both effectively.	3
8	An e-commerce platform shows inconsistent behavior across browsers. Apply testing principles to design a strategy for cross-browser testing. Explain how defects originate in such scenarios. Illustrate how systematic testing improves reliability. Suggest tools and techniques for better coverage.	3
9	A payroll system produces incorrect salary calculations due to logical errors in code. Apply defect classification techniques to categorize these defects. Explain how understanding defect classes improves testing. Illustrate how such defects can be prevented in future releases. Suggest improvements in testing strategy.	3
10	A software team skips proper test case design and directly executes random tests. Apply testing as an engineering activity to redesign the testing process. Explain the importance of structured test case design. Illustrate how this improves defect detection. Suggest suitable techniques to enhance test effectiveness.	3

11	A healthcare application shows data inconsistency across modules. Apply testing principles to analyze the issue. Explain how defects may originate during development. Illustrate how systematic testing ensures data integrity. Suggest improvements in defect tracking and reporting.	3
12	A company faces repeated defects in its software releases despite testing efforts. Apply the concept of defect repository to analyze the issue. Explain how repository data supports test design. Illustrate how defect patterns can be identified. Suggest improvements to prevent recurring defects.	3
13	A web-based learning system fails during peak usage due to untested scenarios. Apply software testing principles to identify gaps in testing. Explain how test planning and coverage impact performance. Illustrate how better test strategies improve system reliability. Suggest enhancements to avoid such failures.	3
14	A team observes that testers are not involved in early development stages. Apply the role of tester in SDLC to improve this situation. Explain how early involvement impacts quality. Illustrate benefits using a practical example. Suggest changes in team collaboration practices.	3
15	A system produces correct outputs in some cases but fails in others due to incomplete test coverage. Apply testing principles to identify gaps. Explain how proper coverage ensures reliability. Illustrate how to improve test case design. Suggest techniques for better validation.	3
16	A logistics application shows delay in defect identification due to poor reporting practices. Apply testing concepts to improve defect reporting. Explain how clear definitions of defects help in tracking. Illustrate how structured reports enhance communication. Suggest improvements in reporting standards.	3
17	A software team struggles to maintain consistency in testing across modules. Apply process-based testing approaches to solve the issue. Explain how standardized processes improve quality. Illustrate your approach with an example. Suggest methods to ensure consistency.	3

18	A system fails after deployment due to untested edge cases. Apply testing principles to identify why such cases were missed. Explain how boundary testing helps in such scenarios. Illustrate how to include edge cases in testing. Suggest improvements in test design.	3
19	A company lacks proper documentation of defects, making it difficult to track issues. Apply defect repository concepts to address this problem. Explain how documentation improves defect management. Illustrate how structured repositories support analysis. Suggest improvements in documentation practices.	3
20	A banking system shows repeated failures in transaction processing due to improper testing practices. Apply software testing principles to redesign the testing approach. Explain how structured testing improves quality. Illustrate your solution with examples. Suggest techniques to enhance defect detection.	3

Engineer to Excel

SAVEETHA

INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Accredited by NAAC A Grade

Code of Conduct:

I Mathusri P - 192311363 *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: Mathusri P

Upgrade
Search
ENG IN

CSA3714 Software Testing

CO1_AT1_Descriptive Test

Mathusri P ①
192311363
1, 6, 13, 14, 16

92/100

① Testing as an Engineering Activity.

(introduction:)

* Software testing as an engineering activity is a systematic process of planning, designing, executing and evaluating tests. cases to identify defects and improve software quality.. it replaces random testing with a structured and repeatable approach.

(Systematic Test Design.)

→ Prepare a test plan based on the requirements
→ Design test cases with inputs and expected outputs
→ include positive, negative and boundary test cases.
→ Priorotize critical banking operations such as withdrawal. and fund transfer.
→ Record test result and report defects.

(Introduction of Tools and statements.)

* Selenium: automates web application testing
* Jira: Records and tracks defects
* JMeter:- Test application performance under heavy loads
* Testing standards :- follow documented testing procedures and standards such as ISO/IEC/IEEE 29119.

* (Techniques to Replace Ad-hoc Testing)

* Equivalence partitioning: Divides inputs into valid and inv groups.
* Boundary value Analysis: Test values at anand around i/p limit.
* Decision Table testing: Tests combinations of conditions
* Regression Testing: checks whether software change affect existing

* (Example:) For a ₹5000 bank transfer, verify that the sender's balance decreases & the receiver balance increases by 5,000 also test insufficient balance & duplicate transfer
* (Conclusion:) Structure testing improves defect detection, reliability, & software quality

---

⑥ Testing a Mobile Banking Application.

* (Introduction:) Systematic testing helps identify the cause of application crashes

* (Testing approach)
→ check crash logs and network errors
→ Perform unit and integration testing
→ Test fund transfer under different network connections con conditions
→ Perform performance and regression testing.

Example:) if the application crashes during a ₹1,000 transfer, verify that money is not deducted twice

(Conclusion:) Root-cause analysis and systematic testing reduce crashes and improve transaction reliability.

(13) Performance Testing of a learning System.

(Introduction:)

* Performance testing checks an application that the performance of the testing of a learning system. that the application is working stable like stability and responsiveness under heavy useage.

(Testing approach.)

* The Testing approchs are:

① Identify peak users load
② Perform load, stress and edurance testing
③ Measure response time and error rate
④ use JMeter to simulate concurrent users.

(Example:)

* Simulate 1,000 students logging in simulataneously to identify server delays.

* (Conclusion:-)
  * proper performance testing prevents failure during peak useage and improves reliability.

(14) Early involvement of Testers in SDLC.

- (Introduction:) Tester should participate in every SDLC phase to identify defects earlier. con early.
- (Roles of testers)
  → Requirements: Identify unclear requirement
  → Design: prepare test scenario.
  → Development: Support early testing.
  → Testing: Execute test cases and report defects
  → Maintenance: Perform regression testing.
- (Example:) A tester identifies missing requirements for duplicate bank transfer before coding begines
- (Conclusion:)

Early testers involvement reduces defects, network, and development costs.

Improving Defects Reporting.

Introduction: Improving Defects Reporting is Defect Reporting Documents software problems so developers can reproduce and fix them.

* A defect report includes.
  - → Defect ID and title
  - → steps to reproduce.
  - → Expected and actual results.
  - → Severity and priority
  - → Screenshots and logs.

* Example:

  If a logistics application shows the wrong delivery date, the tester records the expected date, actual date and, steps to reproduce the issue.

* Components of a Defect Reports. (and importance to clear)
  - Errors: a human mistake during development
  - Defects: A flaw in the software.
  - Failure: The software does not produce the expected output

* (Improvement in Reporting).

→ use a standard reporting template

→ maintain a centralized defect respository.

→ Assign unique IDs to defects.

→ update defects status regularly

→ Perform root - cause analysis to prevent repeated defects.

* (Conclusion.)

* clear and structured defect reporting improves communication, reduces delays in defects identification. and enhances software quality.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25%	Demonstrates thorough, accurate understanding of concepts	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor understanding; major misconceptions
Explanation	25%	Detailed, well-explained answers with relevant examples	Clear explanation with adequate details	Limited explanation; lacks depth	Poor or unclear explanation
Application	20%	Effectively applies concepts with strong analytical ability	Applies concepts with minor errors	Limited application with weak analysis	No application or incorrect analysis
Organization	15%	Well-structured answers with logical flow and coherence	Organized with minor issues in flow	Basic structure, lacks clarity	Poor organization, incoherent answers
Accuracy	10%	Completely accurate and covers all required points	Mostly accurate with minor omissions	Partially correct with missing points	Incorrect answers with major gaps
Clarity	5%	Clear, neat, professional presentation	Understandable with minor issues	Limited clarity, readability issues	Poorly written, difficult to understand